ResearchMind

Project & Viva Guide

60 questions with answers. NLP concepts, formulas, implementation details, and a complete real-paper demonstration.

STUDY MAP

1. Your project in one minute
2. Architecture and data flow
3. NLP concepts you should understand
4. Implementation truth: what to say in viva
5. Viva questions: overview and architecture
6. Viva questions: NLP and retrieval
7. Viva questions: evidence and research reasoning
8. Viva questions: evaluation, security, and engineering
9. Real paper and demonstration script
10. Troubleshooting and examiner follow-ups
11. Revision sheet and suggested evaluation plan
12. Sources and further reading

Read Sections 1-4 first. Practise the questions in Sections 5-8. Rehearse Section 9 with the original RAG paper. Use Sections 10-12 for troubleshooting, final revision, and references.

This guide distinguishes code capabilities from the current local demo. It does not claim that example metrics or generated research proposals are measured scientific results.

1. Your project in one minute

ResearchMind helps a researcher organize NLP papers, inspect source passages, search a project corpus, review candidate limitations, formulate hypotheses and experiment proposals, and export manuscripts with citations. Its central design goal is traceability: keep the paper and passage behind a claim available for inspection.

Say: "My project is a research-support workspace for NLP literature. It turns uploaded or discovered documents into searchable passages and structured records. It combines retrieval, scientific entity extraction, evidence provenance, and a staged research workflow. It supports drafting and experiment tracking, while keeping human review necessary."

Problem: papers are numerous, manual comparison is slow, and generated summaries can contain unsupported statements. The project brings document organization, retrieval, evidence inspection, and report preparation into one workflow.

Input: a research question, searchable PDFs, stored literature metadata, documentation/source archives, or a public GitHub repository. Output: papers and chunks, extracted entities and relations, source-linked claims, candidate gaps and hypotheses, experiment specifications and supplied run records, and manuscript exports.

Contribution: integration and evidence-oriented workflow engineering. The code uses existing retrieval and NLP techniques. Do not claim that it invents RAG, BM25, transformers, or an independently validated gap-discovery algorithm.

2. Architecture and data flow

Frontend: React, TypeScript, Vite, React Router, TanStack React Query, Tailwind, and reusable UI components. The browser collects input, calls the API, displays loading/errors, and renders results and graph/chart views.

Backend: FastAPI routes validate requests and enforce authentication/access. Services coordinate research operations. Repositories read/write SQLAlchemy models. NLP and retrieval modules process documents. Workflow handlers perform staged jobs.

Local runtime: SQLite, in-process asyncio jobs, fakeredis, demo synthesis, and lexical-hash embeddings. SQLite discovery searches a bundled metadata corpus. Discovered records can be indexed from abstracts. Uploaded PDFs provide fuller text.

Production-oriented path: PostgreSQL/pgvector, Redis and ARQ workers, external academic discovery APIs, and optional supported generation/embedding models. Presence in the repository does not prove that a deployment or optional model has been tested.

Data flow: question and project -> discovery/upload -> PDF extraction -> sections -> overlapping chunks -> vectors/entities/relations -> retrieval -> synthesis and source-linked claims -> gap candidates -> hypotheses -> experiment proposals -> verification -> reports.

Important separation: Dashboard PDF review is a source-specific review and draft. It does not automatically execute all corpus research stages. Run the project pipeline to populate the broader research workflow.

Core records: User owns ResearchProject; a project has ResearchQuestion and Paper records; a paper has sections, chunks, and authors. Entities, relationships, claims, and evidence links represent extracted knowledge. Gaps, hypotheses, experiments/runs, reports/citations, and jobs represent research artifacts and workflow state.

3. NLP concepts you should understand

Text preparation, tokenization, and chunking

Tokenization splits text into processing units. This project's BM25 path uses lowercase alphanumeric regex tokens, whereas a trained transformer typically uses its own subword tokenizer. A PDF page, a sentence, a word, a subword token, and a retrieval chunk are different units.

PDF text extraction recovers characters and layout-derived blocks. It is not OCR. Two-column reading order, ligatures, line wrapping, tables, and headers can damage extracted text. The corrected source reviewer normalizes Unicode, joins wrapped lines, and retains physical PDF page numbers for findings.

Chunking divides long text into smaller retrievable units. Defaults in the extraction pipeline are 512 estimated tokens with 64 estimated tokens of overlap. Token count is approximated with characters, not computed using a model tokenizer. Whole-sentence boundaries make actual sizes variable.

Overlap helps preserve context across boundaries but increases storage and repeated evidence. Small chunks improve localization but can omit context; large chunks preserve context but introduce irrelevant text. Tune sizes on an annotated retrieval task rather than assuming the defaults are optimal.

Stopword removal can reduce common-word noise but can also delete meaningful words. Stemming removes word endings heuristically; lemmatization maps words to dictionary forms. Do not claim that the BM25 tokenization path performs stemming or lemmatization: it currently uses regex tokenization.

Sparse retrieval and BM25

Sparse retrieval uses lexical terms. TF measures term occurrence; IDF reduces the contribution of common terms. TF-IDF is a useful conceptual baseline, but the implemented sparse retriever uses BM25Okapi.

BM25(q,d) = sum over query terms t of IDF(t) * f(t,d)*(k1+1) / [f(t,d)+k1*(1-b+b*|d|/avgdl)]. Here d is a chunk, f is term frequency, and avgdl is mean chunk length. k1 controls frequency saturation; b controls length normalization. This code uses k1=1.5 and b=0.75. Strength: exact terminology. Weakness: synonyms and paraphrases. Reference: S3.

Embeddings and cosine similarity

An embedding represents text as a numeric vector. A trained sentence encoder can place semantically related passages near one another; independent passage embeddings can be stored and reused. Sentence-BERT is an influential example of this approach, not a claim that this local demo runs that exact model. Reference: S2.

cosine(q,d) = dot(q,d) / (norm(q)*norm(d)). It compares vector direction. A high cosine score measures similarity under the chosen representation; it does not prove that a passage entails a claim.

The default trained-model configuration names BAAI/bge-small-en-v1.5 with 384 dimensions. The local entrypoint selects lexical-hash instead: tokens are deterministically mapped to buckets, counts are accumulated, and vectors are normalized. Hash collisions can merge unrelated tokens. This is a reproducible lexical fallback, not learned semantic understanding.

Hybrid retrieval, rank fusion, and reranking

Hybrid retrieval combines lexical and vector-based candidate lists. This implementation uses weighted Reciprocal Rank Fusion: score(d)=sum over lists i of weight_i/(60+rank_i(d)), with rank counted from 1. Absent documents contribute zero from that list. Default weights are 0.5 each. Combining ranks avoids directly adding incompatible raw score scales. Reference: S4.

For example, a chunk ranked 1 in BM25 and 3 in the other list receives 0.5/61 + 0.5/63, approximately 0.01613. That is a ranking score, not 1.613% factual confidence.

A bi-encoder embeds queries and passages independently and supports fast vector lookup. A cross-encoder scores query-passage pairs jointly and is usually more expensive. The optional reranker is cross-encoder/ms-marco-MiniLM-L-6-v2. Local lexical-hash mode skips loading it; inspect the returned reranked flag.

Entities, relations, and keyphrases

Scientific NER extracts labels such as MODEL, DATASET, TASK, METRIC, METHOD, BENCHMARK, and ORGANIZATION. The code primarily uses scientific regex patterns, with optional spaCy general entity recognition. A mention of BART or F1 is an extracted entity, not a finding that BART achieved a particular F1 score.

Relation extraction uses trigger patterns and nearby entity pairs, such as evaluated_on, compared_with, improves, and based_on. Example: "BART was evaluated on SQuAD" suggests a model-to-dataset relation. Proximity and triggers can attach the wrong entities or miss negation; inspect the retained evidence.

Keyphrase extraction supports optional KeyBERT and a local n-gram scoring fallback. A keyphrase summarizes a topic; an entity assigns a type to a mention; a claim asserts something that can be checked. These outputs have different purposes.

RAG, transformers, and generation

Retrieval-Augmented Generation retrieves external information before generating text. The original RAG work combines parametric knowledge with retrieved non-parametric memory. This project applies retrieval and evidence organization to scientific documents; uploading the RAG paper does not reproduce its training procedure. Reference: S1.

A transformer uses attention to relate positions in a sequence. Scaled dot-product attention is softmax(QK^T/sqrt(d_k))V. Multi-head attention learns multiple relation patterns. The project consumes optional pretrained models/providers rather than training a transformer from scratch. Reference: S5.

Extractive output selects or preserves source text. Abstractive output generates a paraphrase or synthesis. Local source drafts preserve inspected content and use structured templates/review rules. Configured generation providers can synthesize sections. A relevant citation can still fail to support the precise generated statement.

Provenance, entailment, and research reasoning

Provenance connects a result to its origin: paper, section, chunk, and page metadata. It enables auditability. Local verification uses verbatim occurrence in linked evidence for supported status; without a match it returns insufficient evidence. This checks source occurrence, not independent truth or robust logical entailment.

A limitation describes a constraint; future work is an author's proposed direction; a research gap is a defensible unanswered question; a hypothesis is a testable proposition. They are not interchangeable. The source reviewer now labels future work separately and reports missing keyword matches as manual documentation checks.

4. Implementation truth: what to say in viva

Running locally: SQLite stores records; in-process jobs execute stages; discovery uses the bundled corpus; BM25 and lexical-hash similarity support retrieval; demo drafting and verbatim support checks avoid requiring paid providers.

Optional/configured: trained semantic embeddings, cross-encoder reranking, external generation providers, PostgreSQL vector search, Redis/ARQ, and academic API discovery. Show actual configuration and execution results before claiming these are active.

Not established by the current demo: autonomous invention of scientifically novel gaps, independently correct claim verification, calibrated confidence, fully automatic publication-quality papers, real model training, or reproduction of external paper results.

Experiment runs store supplied metrics; default form values are examples. Some explanation panels and score/runtime fallbacks are templates. Never describe them as measured evidence. The report exporter generates .tex, .md, and .bib; the backend does not compile the LaTeX to PDF.

Backend regression tests passed in the preceding correction session, including the real RAG paper's upload/search/export and ablation-documentation checks. These software tests validate behavior, not retrieval accuracy, scientific novelty, or statistical performance.

5. Viva questions: overview and architecture

Q1. What is the project objective?

To support literature-based research with organized papers, retrievable evidence, traceable claims, structured research artifacts, and editable manuscript exports. Human review remains part of the workflow.

Q2. Why is this an NLP project?

It processes scientific language through tokenization, passage segmentation, lexical/vector retrieval, entity/keyphrase/relation extraction, source review, and optional language generation.

Q3. How is it different from a chatbot?

It persists project artifacts and evidence links across a staged workflow, including plans, papers, hypotheses, experiments, jobs, and reports. A chatbot interface alone does not provide these records.

Q4. What is your actual contribution?

The integration of document processing, evidence inspection, workflow orchestration, and report preparation, with explicit fallback behavior. State only personal contributions you can substantiate from your own work history.

Q5. Why React and FastAPI?

React supports interactive component-based views; TypeScript helps detect interface mismatches. FastAPI provides typed API validation, async endpoints, dependency injection, and interactive API documentation.

Q6. Explain one request end to end.

The user selects a project and query. React calls retrieval/search with authentication. The backend validates project access, ranks scoped chunks, adds source metadata, and returns evidence. The UI displays passages and scores.

Q7. Why services and repositories?

Services coordinate operations; repositories isolate database access. This separation helps test logic without coupling every route to SQL details and makes changes easier to localize.

Q8. Why store both text and vectors?

Vectors support similarity ranking; text supports inspection, generation context, and provenance. A vector alone cannot explain the retrieved evidence to a researcher.

Q9. Why use background jobs?

Document processing and multistage research can outlast a typical request. Jobs expose progress, failure state, and stage information. Local execution is in-process; the production path supports queue workers.

Q10. What happens if a stage fails?

The job should record its failed stage and error rather than claiming completion. Inspect job details and logs, address the cause, and rerun deliberately. Repeated runs can add artifacts.

6. Viva questions: NLP and retrieval

Q11. What is a corpus here?

The collection of papers and passages available to a project. The bundled discovery snapshot and an individual project's ingested corpus are distinct collections.

Q12. What preprocessing do you actually implement?

PDF extraction, section/chunk construction, lowercasing and regex tokens for BM25, and normalization for source review. Do not claim every standard preprocessing technique is present.

Q13. Why chunk instead of searching entire papers?

Passages localize evidence and reduce irrelevant context. Whole-paper representations can hide a useful paragraph inside a long document. Chunk boundaries can still break dependencies.

Q14. Why overlap chunks?

It reduces boundary-related information loss. It also duplicates context, so overlapping results should not be counted as independent supporting sources.

Q15. Are the 512-token chunks exact?

No. The implementation estimates tokens from character count and tries to retain sentences. Actual sizes vary; use a real tokenizer if strict model limits are required.

Q16. Explain BM25 in simple language.

It rewards query terms in a chunk, discounts common terms, limits gains from repetition, and adjusts for chunk length. The formula and current constants are in Section 3.

Q17. How is BM25 different from TF-IDF?

BM25 adds explicit frequency saturation and length normalization to a probabilistic ranking family. TF-IDF names a broader term weighting approach. They are related, not identical implementations.

Q18. Why is dense retrieval useful?

A trained encoder can retrieve passages whose wording differs from a query. This is an optional semantic path here; local lexical hashing does not provide the same semantic capability.

Q19. Does a 384-dimensional vector mean 384 words?

No. It means 384 numeric coordinates. Learned coordinates are representation features; hashed coordinates are token buckets. Neither coordinate is a stored sentence.

Q20. What is cosine similarity?

A normalized dot product measuring vector direction. It supports ranking under a representation, but similarity is not logical support or factual correctness.

Q21. Why combine retrieval methods?

Different methods can miss different passages. Fusion can improve candidate coverage, but an improvement must be measured against single-method baselines on labeled queries.

Q22. What is RRF?

It combines ranked lists using reciprocal rank contributions. This implementation weights each list and uses a rank offset of 60. It does not interpret the result as a probability.

Q23. What does top-k mean?

The number of results returned. Increasing k can improve coverage but add irrelevant context and processing cost. Candidate retrieval and final returned k need not be identical.

Q24. Retrieval versus reranking?

Retrieval finds candidates efficiently; reranking reorders a smaller set using a stronger scorer when available. Requesting reranking does not prove a cross-encoder ran: check the result flag.

Q25. How does NER work here?

Scientific term patterns produce typed mentions, and optional spaCy recognition can supplement organization/product mentions. This is not a newly trained scientific NER model.

Q26. What is relation extraction?

It connects entities using a relation label and evidence. The current trigger/window approach is interpretable but vulnerable to attachment mistakes, negation, and long-distance relations.

Q27. Is the knowledge graph a graph neural network?

No. It is a graph representation and visualization of stored extracted entities and relations. Displaying nodes and edges does not imply message passing or GNN training.

Q28. What is a keyphrase?

A phrase chosen to characterize a document's topic. Optional embedding-based extraction and a local scoring fallback are supported. A high-ranked phrase is not a verified claim.

Q29. What is RAG versus fine-tuning?

Retrieval adds external context at inference/workflow time. Fine-tuning changes model parameters using training data. This app's document upload/indexing does not fine-tune a language model.

Q30. What is hallucination, and does retrieval eliminate it?

In this setting, an unsupported or misleading generated statement. Retrieval can supply relevant sources, but bad retrieval, contradictory evidence, or incorrect synthesis can still produce unsupported text.

7. Viva questions: evidence and research reasoning

Q31. What does evidence provenance mean?

The retained link between a claim/result and its source paper and passage metadata. Demonstrate it by opening the evidence and comparing it with the PDF.

Q32. Is supported the same as true?

No. In local mode it can mean a verbatim occurrence in linked text. The source itself could be mistaken; scientific truth and independent replication require stronger evaluation.

Q33. How are contradictions handled?

The data model/UI can represent contradicted status. Local verbatim verification does not implement general contradiction reasoning. Do not promise all four categories will populate on every dataset.

Q34. What if the claim has no evidence link?

Treat it as insufficient evidence and inspect upstream extraction/retrieval. A generated statement without a mapped passage does not demonstrate passage-level provenance.

Q35. Why is similarity not entailment?

Two sentences can discuss the same topic while asserting opposite outcomes. Similarity ranks related text; entailment asks whether the evidence supports the particular assertion.

Q36. What is a research gap?

A justified unresolved question grounded in literature scope and evidence. It requires checking whether existing work already addresses the issue; a keyword match alone cannot establish novelty.

Q37. What was wrong with the earlier source review?

It split at PDF line breaks, treated words such as however as limitations, missed the plural ablations, and inferred missing documentation from absent keywords. These caused fragments and false findings.

Q38. How was it corrected?

The source reviewer preserves PDF blocks, normalizes text, includes page numbers, separates future work, and makes documentation checks cautious. Tests include the official RAG paper. It remains a heuristic reviewer.

Q39. What is a hypothesis?

A testable proposition connecting a proposed intervention with an outcome under defined conditions. Example proposal: multilingual retrieval improves evidence recall on a specified multilingual query set. This is a proposal, not an observed result.

Q40. What is an ablation study?

A comparison that removes or changes components under controlled conditions to assess their contribution. RAG's existing ablation discussion is documentation to inspect, not a newly discovered omission.

Q41. What is reproducibility?

Providing sufficient data access, preprocessing, code, dependencies, configuration, seeds, and evaluation details to repeat an experiment. A GitHub link is useful evidence of availability, not proof of full reproducibility.

Q42. Can your app generate a publishable paper automatically?

It prepares a source-based draft and exports citations/LaTeX. Authors must check attribution, evidence, methods, experiments, references, and formatting. A generated manuscript is not automatically publishable research.

8. Viva questions: evaluation, security, and engineering

Q43. How would you evaluate retrieval?

Create representative queries with independently labeled relevant passages. Report Precision@k, Recall@k, MRR and/or nDCG, plus latency and failure cases. Keep tuning and held-out test queries separate.

Q44. Define Precision@k and Recall@k.

Precision@k = relevant results in top k / k when k results are returned. Recall@k = relevant results in top k / all relevant results in the judged collection. Define how fewer-than-k results and unjudged passages are handled.

Q45. Define MRR and nDCG.

MRR averages 1/rank of the first relevant result, using zero when none is found. nDCG compares a graded, rank-discounted relevance gain with the ideal ranking. Specify gain and cutoff conventions.

Q46. What does F1 measure?

F1 = 2PR/(P+R), with an explicit convention when the denominator is zero. Define whether P/R refer to entities, tokens, documents, or another unit; these are not interchangeable tasks.

Q47. How would you evaluate generation and citations?

Annotate factual statements, supporting citations, contradictions, and unsupported claims. Assess support and source coverage separately from fluency. Human review and task-specific criteria are stronger than a single generic overlap score.

Q48. What baselines and ablations would you use?

Compare BM25, trained dense retrieval, and fusion; separately evaluate optional reranking and chunk settings. Keep corpus, queries, labels, and hardware fixed. Do not call lexical hashing a trained dense baseline.

Q49. Have you proved the project outperforms other systems?

Not from workflow tests or example metrics. A defensible comparison needs a versioned labeled dataset, real outputs, documented scripts, fair baselines, and uncertainty. Audit existing result files before citing numbers.

Q50. Do 90 passing tests mean 90% accuracy?

No. The previous test run contained 90 passing software tests. Test count is unrelated to an NLP accuracy percentage. State the date/scope of a test run rather than implying a permanent guarantee.

Q51. How do you avoid data leakage?

Separate training/tuning and evaluation sets, fix corpus versions, deduplicate shared documents, and avoid tuning on held-out labels. Ensure generated reports do not treat demonstration metrics as real evaluation results.

Q52. What security controls are relevant?

Authentication, password hashing, project ownership checks, bounded file inputs, secret/dependency exclusions for inspected archives, and treating source text as untrusted data. These are controls to test, not a claim of complete security.

Q53. Authentication versus authorization?

Authentication identifies a user; authorization checks whether that user can access a project or artifact. A valid token alone should not permit reading another user's documents.

Q54. What is prompt injection here?

An uploaded document may contain instructions aimed at the model. Treat it as source data and avoid obeying embedded commands. Prompt wording alone is not a complete defense; inspect outputs and isolate tool permissions.

Q55. Does the app execute uploaded code?

The manuscript inspection flow reads source/documentation without executing it. Experiment-run recording stores supplied measurements; it is not a general code execution or training service.

Q56. Why SQLite locally and PostgreSQL in production?

SQLite reduces setup for a single-machine demo. PostgreSQL supports the intended multiuser/vector deployment path. Database choice does not itself make the NLP more accurate.

Q57. What are the main scaling bottlenecks?

PDF parsing, embedding inference, repeated index construction, database volume, model/provider latency, and job concurrency. Profile before optimizing. The current BM25 search rebuilds an in-memory index rather than proving a production-scale persistent index.

Q58. What caused the Literature Explorer failure?

The UI requested 200 projects while the backend allowed 100. Validation rejected the request, leaving the dropdown empty and discovery disabled. The request limit, default selection, and visible errors were corrected.

Q59. What are the largest limitations?

Extraction noise, heuristic review/extraction, incomplete metadata, local lexical semantics, abstract-only discovered evidence, optional-model dependencies, uncalibrated scores, and no independent proof of generated novelty or performance.

Q60. What is your future scope?

Build labeled evaluation sets, improve PDF/table handling and precise page provenance, add calibrated entailment checks, strengthen graph extraction and search scalability, test deployed model paths, and implement controlled experiments with reproducible artifacts. Label these as proposed improvements.

9. Real paper and demonstration script

Use Patrick Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020. The unmodified official proceedings PDF is saved as data/demo-papers/lewis_2020_rag.pdf. It has 16 searchable pages and is under the manuscript upload limit. It is legitimate research, not the synthetic VeriRAG example. Official source: S1.

Optional companion papers already downloaded: data/demo-papers/asai_2023_self_rag.pdf and data/demo-papers/min_2023_factscore.pdf. Their official records are listed in Section 12. Keep original authorship and sources when discussing all papers.

Before presenting: start backend and frontend, open http://localhost:5173, and sign in. Current demo credentials are researcher@example.com / ResearchPass123. Check Runtime Diagnostics. Avoid showing access tokens or API keys.

1. All Projects: create RAG Factuality Study with the question "How can retrieval and claim-level verification improve factuality in retrieval-augmented generation?" Open the saved project.

2. Dashboard: select it, upload lewis_2020_rag.pdf, enter the paper title, and click Analyze paper and prepare revision. Do not attribute the original research to yourself.

3. Source review: show full passages and PDF page numbers. Check the ablation mention on page 7, code-release reference on page 2, and future-work passage on page 9. These are physical PDF pages in the downloaded proceedings copy.

4. Literature Explorer: select the same project, show Comparison Matrix/Paper Cards and title filtering. Discover with "retrieval augmented generation hallucination". Explain that local discovery searches the bundled corpus.

5. Paper viewer: open the uploaded paper and search "What knowledge source is used for non-parametric memory?" Inspect returned evidence, then compare it with the original PDF.

6. Research Planner: regenerate a plan if needed, inspect it, edit the exposed problem/subquestion fields, and click Approve Plan & Discover.

7. Project details: watch pipeline stages and wait for completed status. A failed job should be explained using its error. Avoid repeatedly launching jobs to refresh the UI.

8. Evidence Provenance: select the project, choose All Claims, search retrieval, inspect a linked passage and status, and compare with the PDF. Other status filters can be empty.

9. Knowledge Graph: select the project, drag/zoom/pan, select a node, and inspect relations. Switch to Analytics & Claims. Explain that the graph represents extracted mentions/relations.

10. Research Gaps: inspect one candidate, click Explore Evidence, and verify its support. Click Generate Hypothesis only after explaining that novelty is not independently established.

11. Hypothesis Lab: read a statement/rationale and click Design Experiment Spec. Explain that variable/falsification panels can contain generic templates.

12. Experiments & Runs: show the proposed configuration and alternative manual Design Experiment form. For recording only, create a clearly named DEMO ONLY experiment and use artificial demo_precision/demo_recall values. Clear placeholder Git commits. Never call these paper reproduction results.

13. IEEE Papers & Reports: click Create corpus review, inspect its sections, and export Markdown, BibTeX, and IEEE .tex. Then demonstrate Analyze source and write draft using the real PDF or the separately labelled synthetic project ZIP.

14. Hybrid Search: query "retrieval augmented generation Wikipedia" and "RAG-Sequence RAG-Token". Show source passages, ranking scores, latency, and actual reranking status.

15. Dashboard and About: show updated counts, timeline/entity mention charts, and bibliography search. Missing upload author/year/venue metadata is possible. Mention frequency is not model performance.

16. Persistence: reload and reopen saved drafts/runs; switch projects to show scope. Sign out to show access control. Demonstrate deletion only on a disposable project.

For the public GitHub manuscript option, use a real small public repository root URL. The app inspects its archive and does not execute code. This operation needs network access, unlike local corpus discovery. IEEE .tex must be compiled separately with IEEEtran, for example in Overleaf.

10. Troubleshooting and examiner follow-ups

Empty claims: select the correct project, clear filters, and check that the research pipeline completed. PDF source review findings are separate from corpus claim records. No evidence link means provenance has not been established for that claim.

Empty literature: verify project selection and visible API errors. Clear the title filter. A new project may have no papers; discovery and upload are different actions. Selecting another project's paper corpus changes results.

Wrong source finding: inspect the full passage and page. New reviews use the corrected reviewer; old saved drafts retain old findings. A mention located in a checklist does not mean the paper is complete.

Bad PDF extraction: check whether text is selectable. Scanned/image PDFs need OCR not provided by this flow. Read columns/tables manually when extraction order is questionable. Source-review block page numbers are more explicit than generic section-level retrieval metadata.

Weak search: use terms present in the corpus, check ingestion, try multiple queries, and inspect local model settings. A hashed representation cannot be assumed to match semantic paraphrases as a trained model would.

No hypotheses or graph: finish the required upstream stages and inspect the job error if it stopped. Do not manufacture entries simply to make a screen nonempty.

Unexpected confidence/runtime: verify how the backend and UI produced it. Local exact-match confidence is a rule output; some UI defaults are placeholders. Neither is an empirically calibrated scientific confidence interval.

If asked something not implemented: say "That is supported only in the optional path" or "That is future scope" and explain how you would evaluate it. Never upgrade an interface label into a claim about actual model behavior.

11. Revision sheet and suggested evaluation plan

Remember these distinctions: extraction versus generation; lexical versus semantic; similarity versus entailment; occurrence versus truth; limitation versus future work versus novel gap; proposed experiment versus measured run; software tests versus NLP evaluation; project corpus versus one report's references.

Know these code locations: frontend/src/pages for screens; backend/app/ingestion for PDF and discovery; backend/app/nlp for chunks/embeddings/extraction; backend/app/retrieval for ranking; backend/app/services/source_review.py for corrected review; verification_service.py for status logic; backend/app/agents for workflow stages; backend/tests for regressions.

Formula revision: cosine=dot/(norms); RRF=sum(weight/(60+rank)); Precision@k=relevant@k/k; Recall@k=relevant@k/all relevant; F1=2PR/(P+R); MRR=mean(first-relevant reciprocal rank). Define edge cases and your unit of evaluation.

Proposed evaluation, not completed results: collect a representative corpus and questions; have reviewers annotate supporting passages and claims; adjudicate disagreements; separate tuning/test sets; compare BM25, real trained dense, fusion, and optional reranking; measure ranking quality, support accuracy, and latency; report uncertainty and error categories. Store corpus/model versions, commands, hardware, seeds, and actual outputs.

For an ablation, change one component under matched conditions. For significance, choose a test consistent with the sampling unit and assumptions; do not claim the UI's generic p-value text performs that test. For reproducibility, keep raw predictions and judgments, not just final averages.

Final presentation sentence: "The system makes evidence and research artifacts easier to organize and inspect. Its current strengths are traceability and workflow integration; scientific conclusions still require source checking and controlled evaluation."

12. Sources and further reading

S1. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. https://arxiv.org/abs/2005.11401 . Official proceedings PDF: https://papers.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

S2. Reimers and Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. https://arxiv.org/abs/1908.10084

S3. Manning, Raghavan, and Schutze. Introduction to Information Retrieval, section on Okapi BM25. https://nlp.stanford.edu/IR-book/html/htmledition/okapi-bm25-a-non-binary-model-1.html

S4. Cormack, Clarke, and Buettcher. Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. SIGIR 2009. https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf

S5. Vaswani et al. Attention Is All You Need. 2017. https://arxiv.org/abs/1706.03762

S6. Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. https://arxiv.org/abs/2310.11511

S7. Min et al. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. https://arxiv.org/abs/2305.14251

Implementation-specific statements were checked against this checkout's source, especially local_main.py, config.py, ingestion_pipeline.py, chunker.py, embeddings.py, ner.py, relations.py, keyphrase.py, hybrid.py, bm25.py, reranker.py, source_review.py, verification_service.py, and the current frontend pages. Future updates may change behavior. Use docs/complete-feature-demo.md for a longer screen-by-screen checklist.